

Plano — Etapa "UI Spec & Protótipo" no pipeline LangNet

Data: 2026-07-16 **Status:** Aprovado — implementação a iniciar pela Fase 1 **Autor:** sessão LangNet (Claude Opus)

Contexto e diagnóstico

Ao revisar o documento de especificação da Quântica (145 páginas), constatou-se que a etapa de **Especificação Funcional** já produz wireframes por caso de uso: cada um dos 23 UCs tem, além do fluxo de interação (Ator ação → Sistema resposta), uma seção "Wireframe da Interface" com um mockup ASCII da tela. Exemplo (UC-001):

```
| Novo Persona |
| Nome: [      ] |
| Canais: [LinkedIn, Instagram, Blog] |
| [ Salvar ] [ Cancelar ] |
```

A seção 7.1 da spec lista **11 telas reais** (Painel de Controle, Gestão de Personas, Editor de Calendário drag-drop, Dashboard de métricas, Funil de Leads, etc.).

Problema: o code generation ignora tudo isso e gera apenas um **executor genérico de Petri Net** — um depurador do pipeline de agentes. As telas reais de negócio nunca são construídas.

O sistema tem duas caras, e só geramos uma

	O que é	De onde vem	Hoje
Cara A — UI operacional	Telas que um humano usa: cadastro de persona, editor de calendário, dashboard, funil de leads (as 11 telas da seção 7.1)	UCs + wireframes + data model	não existe

	O que é	De onde vem	Hoje
Cara B — pipeline agêntico	Workflow autônomo multi-agente orquestrado pela Petri Net	agents.yaml + tasks.yaml + Petri	existe (+ UI de debug)

O pipeline de agentes **permanece** — ele vira o *backend* que as telas reais chamam. Onde há de fato trabalho agêntico (gerar conteúdo, classificar comentários, coletar métricas), o agente executa. Onde é interação humana direta (Salvar Persona, Editar Calendário, Ver Dashboard), a tela chama CRUD determinístico ou dispara a task.

Decisões tomadas com o usuário

- **Formato do mockup:** HTML/CSS renderizado para PNG (não imagem por IA, não só ASCII). O mesmo HTML vira o esqueleto do componente React real no code gen — zero desperdício.
- **Escopo da app final:** UI real de negócio como principal + executor de Petri Net como aba "Admin/Debug".
- **Prosseguimento:** plano detalhado primeiro (este documento), depois Fase 1.

1. O artefato `ui_spec` (JSON estruturado + versionável)

Por tela, o LLM produz:

```
{
  "screens": [{
    "id": "cadastro-persona",
    "name": "Cadastro de Persona",
    "route": "/personas/novo",
    "uc": ["UC-001"],
    "entity": "personas",
    "layout": "form",
    "components": [
      { "type": "text",      "field": "nome",      "label": "Nome", "required": true, "bindTo": "personas.nome" },
      { "type": "textarea", "field": "descricao", "label": "Descrição", "bindTo": "personas.descricao" },
      { "type": "multiselect", "field": "canais",    "label": "Canais", "bindTo": "canais[].nome_canal" }
    ],
    "actions": [
      { "label": "Salvar",   "kind": "task",    "target": "cadastrar_persona_alvo", "primary": true },
      { "label": "Cancelar", "kind": "navigate", "target": "/personas" }
    ]
  }
],
```

```

    "mockup_html": "<div class='...'>...</div>",
    "mockup_png": "data:image/png;base64,..."
  }},
  "navigation": [{"label": "Personas", "route": "/personas"}, {"label": "Calendário", "route": "/calendario"}],
  "action_map": {"cadastrar_persona_alvo": {"kind": "task"}, "editar_calendario": {"kind": "crud"}}
}

```

- **action_map** é o elo com a Cara B: diz quais botões chamam task agêntica e quais chamam CRUD.
- **bindTo** liga campo ↔ coluna do data model (por isso a etapa vem depois do Data Model).

2. Backend — novos arquivos (seguindo o padrão de `data_model.py`)

Arquivo	Papel	Espelha
<code>backend/app/routers/ui_spec.py</code>	Router <code>/api/ui-spec</code> — generate, get, chat/refine, versions, approve	<code>routers/data_model.py</code>
<code>backend/prompts/generate_ui_spec.py</code>	Prompt: spec (UCs+wireframes) + schema_sql → ui_spec JSON + HTML de cada tela	<code>prompts/generate_tasks_yaml.py</code>
<code>backend/agents/langnetui.py</code>	<code>execute_ui_spec_workflow()</code> , <code>_get_llm()</code> , render HTML→PNG	<code>agents/langnetdatamodel.py</code>
3 tabelas novas	<code>ui_spec_sessions</code> , <code>ui_spec_chat_messages</code> , <code>ui_spec_version_history</code>	padrão <code>data_model</code>
<code>main.py</code>	registra <code>ui_spec_router</code>	linha 49-50

Render HTML→PNG: Playwright já instalado. `langnetui.py` sobe cada `mockup_html` num browser headless e salva PNG (base64 na sessão). Barato e determinístico.

3. Code Gen — mudanças em `langnetagents.py`

O bundle `templates/visualtasksexec/` **continua igual** (vira a aba Admin).
Adiciona-se:

- `_generate_business_screens(ui_spec, schema)` — para cada screen, gera um componente React real:
 - formulário com campos do `components[]`, ligados via `bindTo`
 - botão `kind:"task"` → chama ws-server (task agêntica); `kind:"crud"` → chama CRUD determinístico
 - o `mockup_html` é o esqueleto do JSX
 - **Novo `App.jsx` shell**: navegação lateral com as telas de negócio (`navigation[]`) como principal + aba "⚙ Admin / Petri" com o `MainExecutor` atual embutido
 - Arquivos novos: `frontend/src/screens/<Screen>.jsx` por tela + `frontend/src/screens/index.js`
-

4. UI do LangNet — nova etapa visível

- `src/pages/UISpecPage.tsx` (galeria de mockups PNG + editor + chat de refino)
- Rota em `src/App.tsx`: `/project/:projectId/ui-spec`
- Item de menu em `NavigationContext.tsx` (~linha 107) — **entre "Data Model" e "Agentes & Tarefas"**

Pipeline resultante:

Requisitos → Spec → Data Model → [NOVO: UI Spec & Protótipo] → Agent-Task Spec → YAMLS → Petri → Code Gen

5. Ordem de implementação (fases)

1. **Fase 1 — artefato + backend**: tabelas, router, prompt, `langnetui.py` com render PNG. Testável via curl: gera `ui_spec` da Quântica, conferir JSON + PNGs.
2. **Fase 2 — UI LangNet**: página de revisão dos mockups + refino por chat.

3. **Fase 3 — code gen:** `_generate_business_screens` + novo App shell. Gera app real da Quântica, subir, testar tela de Cadastro de Persona chamando o CRUD determinístico.
 4. **Fase 4 — validação E2E** nos 2 projetos + relatório visual.
-

6. Decisões de design resolvidas

- **Chicken-and-egg UI ↔ agentes:** UI Spec roda antes do Agent-Task Spec, mas referencia nomes de task. Resolvido por convenção verbo_objeto (UC-001 "Cadastrar Persona" ↔ task `cadastrar_persona_alvo`) + matching por similaridade no code gen. Não trava a ordem.
 - **Telas sem entidade** (dashboards, relatórios): `kind: "task"` puro, sem `bindTo`.
 - **Anti-timeout:** mockup gerado 1 tela por chamada LLM (mesma estratégia chunked que já funciona no `tasks.yaml`).
-

7. Riscos

- LLM gerar HTML inconsistente → mitigar com few-shot + validação (tem `<form>` ? tem os campos do schema?) + retry, igual no `tasks.yaml`.
 - Volume: Quântica tem ~11 telas × (JSON + HTML + PNG). Chunked resolve.
 - Esforço estimado: ~2 sessões. Fase 1 é a mais pesada (arquitetura); Fase 3 é a mais delicada (assemblagem do React).
-

Referências de código (pontos de integração confirmados)

- Frontend gerado vem de bundle fixo: `backend/agents/templates/visualtasksexec/` com substituição de placeholders (`_render_visualtasksexec_templates` em `langnetagents.py:3549`).
- Padrão de router de etapa: `backend/app/routers/data_model.py` (GenerateRequest, background task, sessions table, chat, versions).
- Registro de routers: `backend/app/main.py:40-57`.
- Menu do pipeline na UI: `src/contexts/NavigationContext.tsx:102-121`.

- Rotas: `src/App.tsx:83-176`.